Scale Run Notes & Quirks

This document tracks environment, hardware, and model quirks encountered while running the scaled (N=1000) ProcessBench evaluation on 2x RTX 3090s (48GB VRAM total).

1. Teacher Model Selection (27B)

- **The Issue**: The base model `google/gemma-2-27b-it` does not include AWQ quantization natively. Loading it in `FP16` format requires ~54GB of VRAM, which causes an Out-of-Memory (OOM) error on our 48GB setup. The exact model ID `gemma-4-26b-a4b-it` referenced in `HANDOFF_SAKSHAM.md` was a local/internal alias that did not exist on Hugging Face.
- **The Fix**: We fell back to a popular community-quantized version: `mbley/google-gemma-2-27b-it-AWQ`.

2. AWQ Data Type Compatibility

- **The Issue**: When launching `vLLM` with the `mbley` AWQ model, it crashes with `ValueError: torch.bfloat16 is not supported for quantization method awq`. The model was saved with `bfloat16` weights, but standard `awq` kernels only support `float16`.
- **The Fix**: Explicitly pass `--dtype float16` and `--quantization awq_marlin` when launching the `vLLM` API server.

3. Gemma-2 Chat Template Bug

- **The Issue**: The `mbley` AWQ model repository contains an older, buggy `chat_template` in its tokenizer configuration. When the `omlx_client.py` sends a single `{"role": "user", "content": prompt}` message to the `/chat/completions` endpoint, Jinja2 crashes with `jinja2.exceptions.TemplateError: Conversation roles must alternate user/assistant/user/assistant/...`.
- **The Fix**: We created a custom, fail-safe `chat_template.jinja` file locally and passed it to `vLLM` using the `--chat-template chat_template.jinja` argument. This overrides the buggy strict validation and successfully processes the client's prompts.

Final vLLM Launch Command:

```
python3 -m vllm.entrypoints.openai.api_server \
    --model unsloth/gemma-2-27b-it-bnb-4bit \
    --quantization bitsandbytes \
    --load-format bitsandbytes \
    --pipeline-parallel-size 2 \
    --port 8080 \
    --api-key sk-local \
    --chat-template chat_template.jinja
```

4. AWQ Marlin & Gemma 2 Logit Soft-Capping Silent Failure

- **The Issue**: The `mbley` AWQ model with `--quantization awq_marlin` appeared to run extremely fast (900+ tokens/s), but generated 100% empty strings (invisible `<pad>` tokens). This is because Gemma-2 introduced a novel "Logit Soft-Capping" technique, which is incompatible with many standard AWQ/Marlin kernels. The kernels exploded into `NaN` values silently, resulting in `0` kept candidate solutions over a 45-minute run.
- **The Fix**: We abandoned the broken AWQ model entirely and migrated to `unsloth/gemma-2-27b-it-bnb-4bit`. We launched vLLM using `--quantization bitsandbytes --load-format bitsandbytes`. This natively supports Gemma-2's architecture without the Marlin kernel bugs.

5. Infinite Timeout Loop at High Concurrency

- **The Issue**: After scaling the `ThreadPoolExecutor` to 32 workers to maximize GPU utilization, the generator pipeline silently stalled at 5%. At maximum load, generating a full 512-token reasoning trace takes the GPU ~138 seconds. However, `omlx_client.py` had a hardcoded `requests` timeout of `120` seconds with an automatic 3-retry fail-safe. The client was silently aborting the connection right before the GPU finished generating the answer, and then retrying the exact same problem from scratch, resulting in an infinite loop of wasted computation.
- **The Fix**: We edited `models/omlx_client.py` and bumped the `timeout` parameter from `120` to `600` seconds (10 minutes) to provide enough headroom for maximum-length generations under heavy concurrent load.

6. Training Phase OOM Prevention

- **The Issue**: The `vLLM` server natively reserves 90% of GPU memory (44GB out of 48GB) to maximize its KV Cache. The master `run_student_ablation.sh` script executes the generation and labeling steps (Steps 2 and 3) by pinging `vLLM`, but then moves straight into local PyTorch training for the 1.5B student model (Step 4). If PyTorch attempts to load the student model, optimizer states, and training batches into the remaining 4GB of VRAM while `vLLM` is still running, it will instantly crash with a `CUDA Out of Memory` error, destroying the multi-hour ablation run.
- **The Fix**: We hot-patched `scripts/run_student_ablation.sh` to explicitly execute `pkill -f vllm.entrypoints.openai.api_server` immediately before Step 4 begins. This gracefully shuts down the teacher model and frees the entire 48GB of VRAM for the student training loop.

7. Hardcoded Port Crash in Labeling Phase

- **The Issue**: Immediately upon finishing the 1.5-hour generation phase, the pipeline crashed during the transition to the Labeling Phase. While the generation script correctly respected our `vLLM` server port (`8080`), `data/label_pipeline.py` had a hardcoded override that forced it to look for the server on port `8000`, resulting in a fatal HTTP 404 error.
- **The Fix**: We verified the 4,000 generation traces were safely written to disk, modified `data/label_pipeline.py` to dynamically fallback to `os.environ.get("OMLX_URL")`, commented out the generation steps in the master script, and seamlessly resumed the pipeline from Step 3 without data loss.

8. Infinite Token Generation Deadlock in Labeling Phase

- **The Issue**: The Labeling phase uses `temperature=0.0`. When `vLLM` served the math scoring prompt to `unsloth/gemma-2-27b-it-bnb-4bit` at temperature 0, the model failed to output an `<end_of_turn>`

EOS token after scoring the step. Instead, it infinitely generated empty spaces and newlines up to the `vLLM` default ceiling of 1024 tokens. At maximum concurrent load, generating 1024 empty tokens for all 8,000 reasoning steps caused a catastrophic deadlock, spiking the labeling ETA from 40 minutes to over 3.5 hours of wasted GPU compute.

- **The Fix**: We killed the deadlocked script, hard-restarted the `vLLM` server to flush the 8,000 stuck requests in its queue, and injected `export OMLX_LABEL_MAX_TOKENS=50` into the master bash script. This forcefully truncates the generation early, returning the pipeline to a blazing fast ~1.5 hours per labeling pass.

9. Google Gemma-4 Architecture & VRAM Constraints

- **The Issue**: We successfully upgraded `vLLM` and `transformers` to their bleeding-edge nightly/main builds in `~/.local` to access the `gemma4` configuration class. We manually hot-patched three bugs in the `vLLM` Python source code (`rope_scaling` empty dict crashes, `tokenizer.all_special_tokens_extended` removal, and multimodal `text_config.vocab_size` nesting). However, we immediately hit a mathematical hardware constraint:
    - The canonical `google/gemma-4-26B-A4B-it` repository is unquantized `bfloat16`. The weights alone consume 52GB.
    - Our dual RTX 3090 hardware has a strict ceiling of 48GB VRAM.
- **The Fork in the Road**:
    - **Option 1 (Official Unquantized via BitsAndBytes):** Shrink the 52GB weights on the fly to 4-bit (14GB) using `bitsandbytes`, retaining the mathematically sound native checkpoints.
    - **Option 2 (Community Pre-Quantized W4A16):** Download a pre-quantized GPTQ/AutoRound repo to natively use ultra-fast execution kernels.
- **The Decision**: We actively rejected Option 2. `vLLM`'s W4A16 Marlin execution kernels hardcode an assertion for the `SiLU` activation function, while Gemma 4's MoE uses a custom `GELU` layer, meaning Option 2 would instantly crash. Furthermore, squeezing fine-grained 26B MoE experts into 4-bit causes catastrophic precision loss, degrading the scientific validity of our research ablation.
- **The Final Hurdle (Catch-22)**: We locked in Option 1, but discovered the nightly build of `vLLM` has explicitly stripped the ability to perform on-the-fly `bitsandbytes` quantization (throwing a hard error when using `--load-format auto`). When forced to use `--load-format bitsandbytes`, `vLLM` assumed the Hugging Face repo was already 4-bit, attempted to blindly load the 50GB unquantized `bfloat16` weights natively into our 48GB VRAM, and instantly triggered a fatal `CUDA Out of Memory` kernel panic.
- **The Resolution**: To bypass the `vLLM` quantization limitation without sacrificing the mathematically sound native weights, we created an offline python script (`scripts/quantize.py`). This script uses pure `transformers` and `BitsAndBytesConfig` to download the 52GB model, cleanly compress it to 4-bit `nf4` in System RAM, and save the compressed format locally (`models/gemma4-bnb-4bit`).
- **The Quantization Script Quirks**: While running the offline quantization script, we hit several library bugs and VRAM leaks that required targeted patching:
    1. **Accelerate Memory Estimator Bug**: `transformers` incorrectly assumed the 14GB 4-bit footprint would require 52GB of VRAM and crashed. We overrode `max_memory={0: "20GB",`

`1: "20GB", "cpu": "200GB"}`.

2. **PyTorch Shard Fragmentation Leak**: Google published the Gemma 4 MoE packed entirely into a single massive 47GB `.safetensors` file. Loading this massive shard caused PyTorch's VRAM garbage collector to lag, resulting in an OOM on GPU 1 at 59% completion. By injecting `llm_int8_enable_fp32_cpu_offload=True` and the `"cpu": "200GB"` device map, we forced the massive 16-bit `lm_head` and the memory spike to safely overflow into our 251GB System RAM instead.
3. **Params4bit CPU Dispatch Bug**: At 100% completion, `accelerate` illegally passed an `_is_hf_initialized` flag to the bitsandbytes layers, causing a `TypeError`. We monkey-patched `bitsandbytes.nn.modules.Params4bit.__new__` to explicitly pop the invalid kwarg.
4. **Tied Weights AttributeError (The Fatal Blow)**: Despite all patches, during the final `model.save_pretrained()` call, `transformers` attempted to un-tie the `lm_head` and `embed_tokens` (which were offloaded to the CPU meta device) and crashed. Our monkey-patch `transformers.modeling_utils.remove_tied_weights_from_state_dict` bypassed the logic, but the deep internal `accelerate.load_offloaded_parameter` function still fundamentally broke when trying to reconstruct the `safetensors` dictionary from the meta device, throwing a hard `AttributeError: weight is not an nn.Module`.

10. The Pivot to Native vLLM CPU Offloading (100% Unquantized)

- **The Final Decision**: We are running the model **100% UNQUANTIZED**. We *wanted* to quantize the model offline using our local `scripts/quantize.py` so it would fit natively in VRAM and run at lightning speed. However, this offline quantization pipeline was fundamentally shattered. Because Google packed the entire 26B architecture into a single massive 47GB `.safetensors` file, the Hugging Face `transformers` conversion logic suffered from catastrophic memory leaks and CPU-offload type-errors that threw uncatchable `AttributeError`s right before saving to disk. Since local quantization was literally impossible, we completely abandoned Option 1.
- **The Execution**: We reverted to pure, mathematically-valid unquantized execution. To physically fit the massive 52GB `bfloat16` model onto our 48GB hardware, we natively launched `vLLM` using the `--cpu-offload-gb 20` argument. This forces `vLLM` to store 32GB of the weights on the GPUs, and spill the remaining 20GB of weights into System RAM.
- **The Latency Tradeoff**: During execution, `vLLM` must stream the 20GB of unquantized parameters over the motherboard's PCIe bus for every single token generation. This is exactly what causes the massive latency penalty (scaling the generation phase to ~4 hours). It is not "the worst of both worlds" (it is not quantized). It is the unavoidable hardware tax for executing the pure, uncorrupted weights without falling back to smaller models.
- **Google's vLLM Architecture Bug**: Even with `--cpu-offload-gb 20`, `vLLM` instantly crashed with a CUDA OOM on GPU 0. We discovered that the official Google `transformers` code inside `modeling_gemma4.py` illegally instantiates its `Gemma4TextExperts` layer using a raw `torch.empty(...)` call. This completely bypassed `vLLM`'s meta device initialization and instantly tried to allocate the full 52GB parameter skeleton natively on the primary GPU!
- **The Final Patch**: We manually hot-patched `~/.local/lib/python3.10/site-packages/transformers/models/gemma4/modeling_gemma4.py`, injecting `device="cpu"` into both massive `torch.empty` allocations. This forced the skeleton to safely build in our 251GB System RAM.
- **Status**: `vLLM` is now successfully booting and streaming the pure unquantized weights over the PCIe bus! While this incurs a massive latency penalty, it cleanly bypasses all quantization bugs and ensures perfect scientific validity for the ablation.

11. The Final Pivot: Native Hugging Face Pipeline (Bypassing vLLM)

- **The vLLM Death Blow**: Exactly 5 minutes after safely loading the weights into System RAM to avoid the `torch.empty` crash, the `vLLM` fallback engine crashed with a hard `ValueError: Trying to run tensor parallelization but the model does not support it yet!`. Because Gemma 4 is a brand new architecture, `vLLM` fundamentally lacks the internal tensor routing logic (`base_model_tp_plan`) required to shard its 3D `Gemma4TextExperts` parameter blocks across multiple GPUs. Since a 52GB model physically cannot execute on a single 24GB GPU without Tensor Parallelism, `vLLM` is structurally incompatible with Gemma 4.
- **The Solution**: We completely abandoned the `vLLM` server architecture. We rewrote `scripts/generate_solutions.py` and `models/device.py` to run the unquantized model using the native Hugging Face `transformers` library. By leveraging `device_map="auto"` alongside `max_memory={0: "22GB", 1: "22GB", "cpu": "200GB"}`, Hugging Face natively shards the unquantized model across both GPUs and explicitly handles the PCIe CPU offloading boundaries without requiring custom Tensor Parallel routing logic.
- **The 23-Day Bottleneck**: Although the native Hugging Face pipeline successfully utilized the GPUs without OOMing, it natively lacks `vLLM`'s Continuous Batching engine. It was generating candidate solutions strictly sequentially (batch size 1). Streaming 20GB over the PCIe bus sequentially for 6,000 sequences was mathematically calculated to take 23 Days.
- **Status**: The native Hugging Face pipeline (`task-2031`) was manually killed to prevent a 23-day lockup.

12. Returning to Pre-Quantized Gemma 2

- **The Final Decision**: The hardware limits of 2x RTX 3090s combined with the architectural flaws of Gemma 4 make it impossible to run the unquantized model within a reasonable timeframe. We are formally abandoning `google/gemma-4-26B-A4B-it`. We are proposing a pivot back to `unsloth/gemma-2-27b-it-bnb-4bit`. This model is fully supported by `vLLM`, natively quantized to 4-bit, perfectly fits in VRAM, and will restore the continuous batching engine to complete Phase 2 in ~20 minutes.